CookMem Tutorial 3

# Single-Threaded Performance

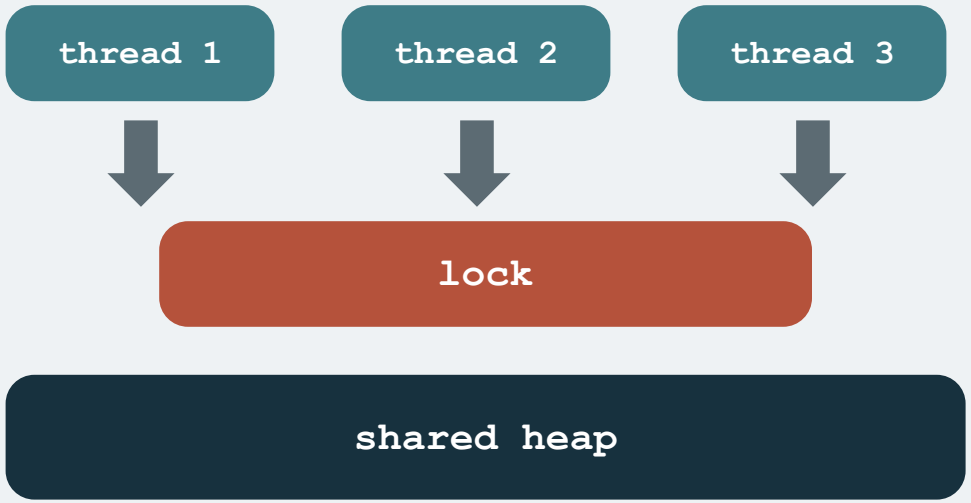
No locks: a memory context for each thread, or each block



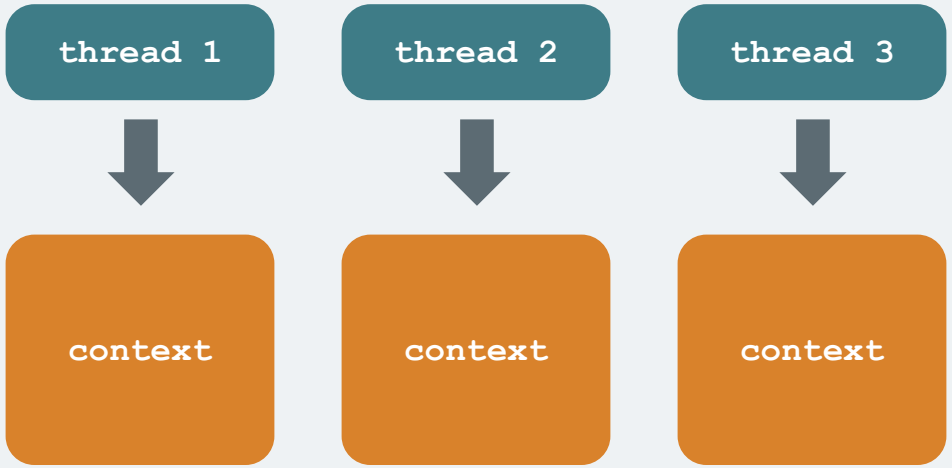
For thread-per-core servers, MPP executors, batch jobs, simulations and game loops

# Why a shared heap costs time

### malloc and free: one shared heap




### CookMem: a context per thread




1. **Every call synchronizes**
   Locks or atomic operations, even when no other thread is waiting.

2. **Caches help, not always**
   Per-thread caches still synchronize when empty, and on cross-thread frees.

3. **A context never locks**
   It belongs to one thread at a time.

# You choose the scope

**Thread scope**

`TLSContextProvider`

Each thread gets its own context. Containers use it without passing allocators around.

**Block scope**

`TLSContextScope`

A block of code gets a context that goes away with the block.

**Any lifetime**

`a context object`

A request, a query or a frame owns its context, as in tutorial 2.




A block's context lives inside its thread, and can share the thread context's cached segments.

# Thread scope and block scope in code

```
struct WorkTag;
typedef cookmem::TLSContextProvider<Context, WorkTag> Provider;
template<class T>
using Alloc = cookmem::Allocator<T, Provider>;

void worker ()
{
    std::vector<int, Alloc<int>> values;    // thread scope
    {
        cookmem::TLSContextScope<Context, WorkTag> scope (true);
        IntMap squares;                     // block scope
    }   // the block's context goes away here
}
```

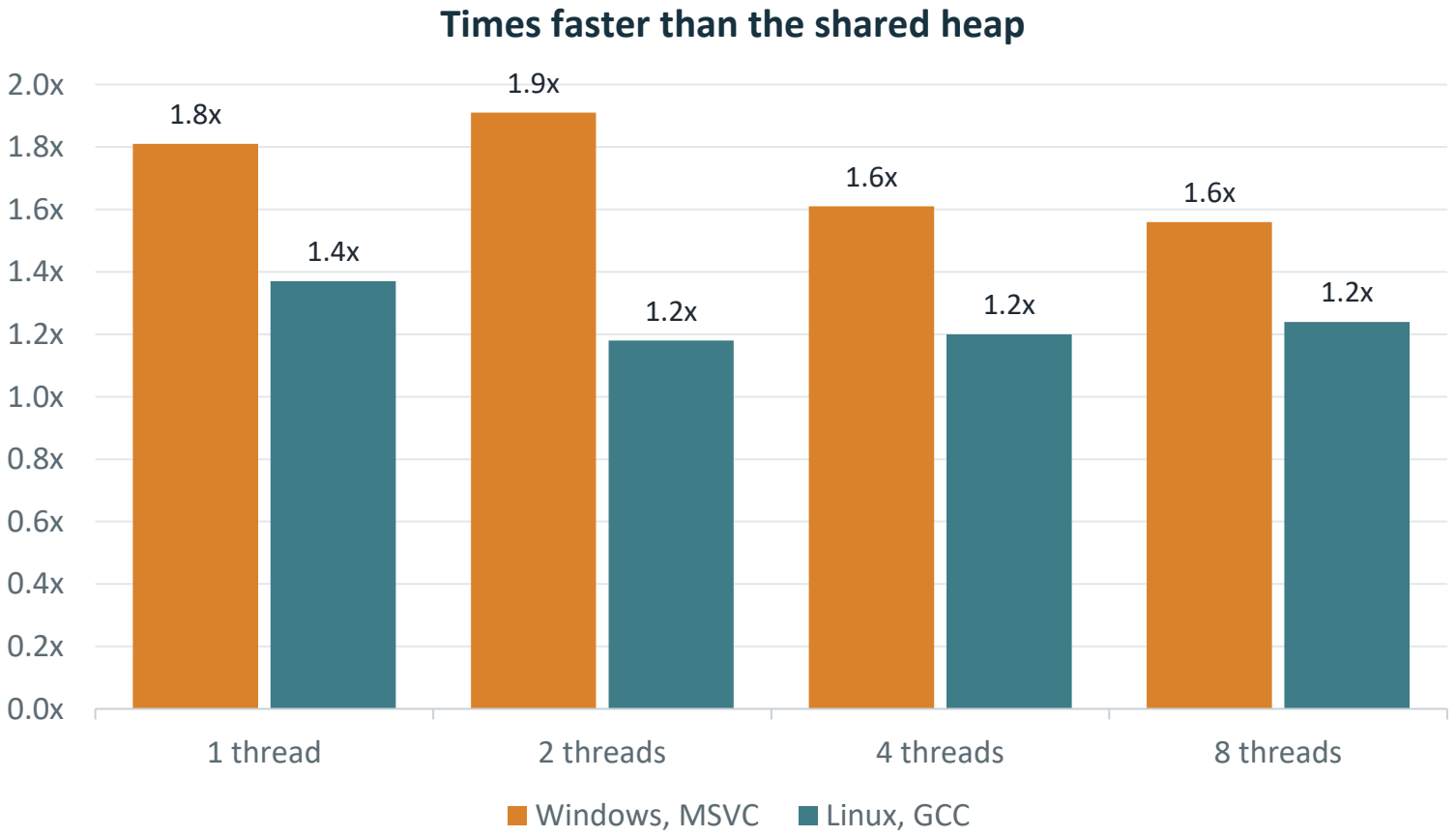
*examples/01_thread_and_block_scope.cpp (shortened)*

1. **Default allocator**
   Uses the thread's active context.

2. **TLSContextScope**
   A context for one block of code.

3. **releaseAllToSystem ()**
   Returns a thread's memory when done.

# Measured: a context per thread




## Windows: 1.6 to 1.9x

The heap synchronizes on every call.

## Linux: 1.1 to 1.4x

glibc's per-thread cache avoids most locking for small sizes.

**Run it on your machine:**
`02_threads_benchmark.cpp`

AMD Ryzen 7 5800H, 8 cores; Windows 11 with MSVC 19.39 (Visual Studio 2022), and Ubuntu 26.04 on WSL2 with GCC 15.2; best of three runs, Release builds.

# Keep the shared step rare

```
// The minimum segment size of the arena, 64 KB by default.
cookmem::MmapArena arena (16 * 1024 * 1024);
cookmem::StatsLogger logger;
Context memCtx (arena, logger);

// ... 100,000 allocations of 64 bytes ...

logger.getStats ().getNumGetSegments ();     // 1
```

*examples/04_segments.cpp (shortened)*

**Reuse segments too:** CachedMemContext keeps released segments, so contexts created again and again do not go back to the system.



# Bonus: bulk release and growing in place

### Bulk release

One releaseAll () instead of one free per object.

tutorial 2

### Grow in place

reallocate () grows into the free memory after an allocation.

5.7x faster

**Reading the README table:** its dlmalloc is built without locks, so it compares the algorithms. The locking advantage shows against your system allocator, as in the benchmark.

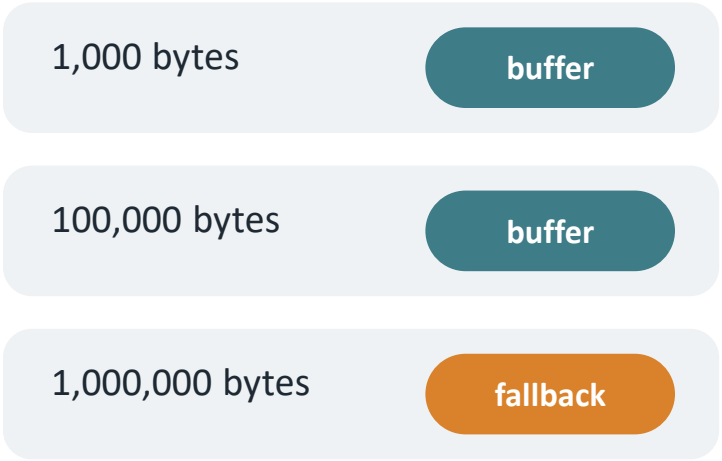
# No system calls on the hot path

```
static char buffer[256 * 1024];          // allocated once
cookmem::FixedArena arena (buffer, sizeof (buffer));
FixedContext fast (arena, logger);

void* scratch = fast.allocate (size);
if (scratch == nullptr)                  // the buffer is full
    scratch = fallback.allocate (size);
// ... the request's work ...
fast.releaseAll ();      // the whole buffer is free again
```

*examples/03_fixed_arena.cpp (shortened)*

**Where each request went**




**Size the buffer for the common case.** A request that does not fit goes to the fallback context, so it still succeeds, just not on the fast path.

# Measure your own workload

1. **A realistic workload:** Your allocation sizes and patterns, not a micro-benchmark.
2. **Your thread count:** Contention depends on how many threads allocate at once.
3. **Release builds, several runs:** Take the best or the median; single runs vary.
4. **Against your system allocator:** That is where the locking advantage shows.

# Pitfalls to avoid

### Sharing a context between threads

Contexts have no locks. Use one per thread.

### Freeing on another thread

Free through the owning context, on its thread.

### Memory kept after the work

Call releaseAllToSystem () when a thread is done.

### A FixedArena that is too small

Keep a fallback context for large requests.

# Summary

1. **No locks**
   A context belongs to one thread at a time, so it never synchronizes.

2. **You choose the scope**
   Thread scope, block scope, or any lifetime you own.

3. **Measure**
   The gain depends on your workload and your system allocator.



Next: Tutorial 4, Debugging Without Valgrind · github.com/coconut2015/cookmem-tutorials